

Privacy-Preserving Probabilistic Data Encoding for IoT Data Analysis

Zakia Zaman ¹, Wanli Xue ², Praveen Gauravaram ², Wen Hu ², and Sanjay K. Jha ²

¹UNSW

²Affiliation not available

September 07, 2023

Abstract

In this research, we have proposed a novel differentially private Bloom Filter data encoding technique, that enables the use of the Deep Learning Algorithm for downstream analysis.

We have used public datasets to evaluate the proposed method.

Privacy-Preserving Probabilistic Data Encoding for IoT Data Analysis

Zakia Zaman, *Student Member, IEEE*, Wanli Xue, *Member, IEEE*, Praveen Gauravaram, *Member, IEEE*,
Wen Hu, *Senior Member, IEEE*, and Sanjay Jha, *Senior Member, IEEE*

Abstract—The widespread prevalence of the Internet of Things (IoT) has emerged as a vital factor in achieving sustainable progress. Providers of IoT services commonly collect user information to analyze using sophisticated Deep Learning (DL) techniques. This enables them to extract valuable insights for business intelligence and enhance their services' quality. However, due to the sensitive nature of this personal data, there's a potential risk of exposing private information due to deliberate or unintentional cyberattacks. To mitigate this risk and protect privacy, experts recommend rapid, effective, and robust data encoding methods for subsequent operations. One widely accepted encoding method is the Differentially Private (DP) Bloom Filter (BF) encoding. Although significant research has examined the privacy-preserving BF-encoding technique, it's uncertain whether this encoded data can be used with various DL algorithms for tasks like classification and future data prediction. Moreover, it's not verified if these encoded datasets can safeguard DL models from Membership Inference Attacks (MIA). In our research, we present a novel approach by combining BF-encoding with Local Differential Privacy (LDP) to encode datasets from different domains, including color images, facial images, and time-series data. Our method outperforms standard BF-encoded data when utilized in DL algorithms for downstream tasks, achieving around $\approx 30\%$ improvement in image classification accuracy compared to previous BF-encoding methods. Additionally, we assess the privacy of the DL model against MIA. The results of the attack simulation indicate that attackers can only make random guesses while trying to expose membership information, with their accuracy being around $\approx 50\%$.

Index Terms—data encoding, bloom filter, privacy-preserving machine learning, privacy, and utility.

I. INTRODUCTION

Ensuring privacy during data analysis has become a major concern for modern data-driven entities like smart cities and smart healthcare systems. These entities gather large volumes of user data to gain insights and generate new business value. For instance, they collect sensor data from Internet of Things (IoT) devices at regular intervals to monitor their customers' health and provide tailored services accordingly [1], [2]. Managing such massive datasets involves resource-intensive tasks, leading organizations to rely on third-party cloud service providers like AWS, Microsoft Azure, and Google Cloud. These providers utilize various Machine Learning (ML) and Deep Learning (DL) algorithms on their infrastructure to perform tasks such as modeling, classification, and prediction [3]. While leveraging the cloud for these critical workloads offers business advantages, it also introduces potential risks such as data breaches, as the privacy safeguards provided by cloud providers may not always be adequate [4], [5]. The cloud

is typically regarded as "honest-but-curious," meaning it guarantees data and model integrity and adheres to established protocols but may covertly seek information for potential business benefits [1], [6]. Therefore, in many cases, organizations prefer privacy preservation techniques at the data source instead of relying solely on data protection methods implemented in the cloud. To accomplish this, they utilize various encryption and encoding methods before making the data available [7]. Numerous contemporary data protection approaches have been suggested, including Fully Homomorphic Encryption (FHE) [8], Data Anonymization [9], and Differential Privacy through noise addition [10]. However, these methods have their limitations. FHE requires specialized hardware and bandwidth support, which increases the overhead on the client's side [11]; data anonymization techniques are inadequate in certain privacy breach scenarios [12], and noise generation and addition methods are considered more efficient but present the challenge of determining the appropriate level of noise, striking a balance between privacy and utility trade-offs [7].

Probabilistic encoding, such as Bloom Filter (BF) encoding in conjunction with Differential Privacy (DP), has attracted considerable attention as one of the data encoding techniques for privacy-preserving data analysis. For instance, RAPPOR [13] has been employed in some of Google's applications to collect privacy-preserving statistical data from client-side strings using random response perturbation. Another study by Xue *et al.* [14] introduced a novel sequence data encoding scheme using Bloom Filter, applying BF encoding to publicly available datasets such as MNIST, Human Activity, and CSI datasets [15]–[17]. They demonstrated that specific machine learning algorithms like SVM, KNN, and Random Forest Classifier could successfully classify the encoded data.

Despite extensive research on BF encoding, it is still uncertain whether this method can effectively handle diverse and complex datasets while giving priority to preserving privacy and being suitable for various deep-learning tasks. Moreover, the privacy of ML/DL models trained using BF-encoded datasets has not been assessed against Membership Inference Attacks (MIA), which involve extracting membership information from the training dataset [18]. Recent studies have also revealed vulnerabilities in BF-encoded datasets, where attackers can reconstruct the Bloom Filter contents given enough computational resources [19]. Niedermeyer *et al.* demonstrated in their work [20] that pattern-based attacks can be carried out on BF-encoded datasets due to the use of cryptographic hash functions like MD5 and SHA-1 in the Double Hashing (DH) technique used for mapping data into

the Bloom Filter. However, Schnell and Borgs [21] showed that using Random Hashing (RH) with a Pseudo-Random Number Generator (PRNG) for generating index positions can prevent such pattern-based attacks. Notably, the state-of-the-art (SOTA) BF-encoding technique mentioned in [13], [14] did not address this specific issue.

This paper proposes a novel BF-encoding technique that encodes different data types, including color-image and time-series data. We adopt the RH technique to enhance the privacy of the BF-encoding process. We then use an LDP algorithm that adds random noise to the encoded data from the Laplace distribution, providing an ϵ -DP guarantee to the encoded dataset. We demonstrate that the CNN-based DL algorithm can efficiently classify and forecast the encoded image and time-series datasets using this approach, which we refer to as BloomCNN. Our proposed BF-encoded image dataset outperforms the SOTA BF-encoded image data when used in the DL algorithm, with an increase of $\approx 30\%$ in image classification accuracy. Additionally, to our knowledge, this is the first study to demonstrate the possibility of forecasting data from BF-encoded time-series data. We assess the privacy risk of our BloomCNN by using the TensorFlow-privacy's [22] MIA utility functions to simulate the attack and generate the privacy report. The privacy analysis results show that the adversaries, in most cases, are making random guesses as the attack accuracy is around 0.5. It means there is a $\approx 50\%$ chance that the attacker will be able to distinguish between a member and a non-member [23].

To summarize, our contributions to this research are:

- This paper proposes a novel BF encoding scheme for privacy-preserving DL-based downstream tasks. We demonstrate that the CNN-based DL model can efficiently classify and forecast using our proposed encoded data with high accuracy compared to the previous BF-encoded dataset.
- We utilize the RH technique and an LDP algorithm in BF-encoding. Our results demonstrate that our proposed BF-encoding scheme, which incorporates LDP, provides better privacy preservation against MIAs than traditional DH-based BF-encoding methods. Furthermore, we show that the LDP algorithm provides an ϵ -DP privacy guarantee to the encoded data. We addressed the issues regarding the proof of ϵ -DP in previous research [14] and provided a new proof of ϵ -DP by perturbing random noise from the Laplace distribution with the original BF-encoded data to ensure DP.
- To demonstrate the effectiveness of BloomCNN, we evaluate it with various datasets, including time series, color images, and facial images. Our results show that BloomCNN significantly improves utility and privacy in DL. Additionally, we utilize the state-of-the-art tool Uniform Manifold Approximation and Projection (UMAP) to validate that our proposed encoding technique preserves the data distribution.

The rest of the paper is organized as follows: Section II discusses the background study and related work. Section III discusses the Threat Model, Section IV describes the proposed

BF-encoding and BloomCNN model, Section V describes the datasets and experiment results, Section VI discusses the impact of different encoding parameters, Section VII shows the performance comparison of our DL model with other state-of-the-art algorithms, and results of MIA on BloomCNN, Section VIII compares the performance of our BF-encoded dataset in the DL model with the state-of-the-art BF-encoding technique, and Section IX makes some concluding remarks on this work.

II. BACKGROUND

This section discusses the necessary background on Bloom Filter, its application in the privacy-preserving domain, Binary Neural Networks, and Local Differential Privacy (LDP).

A. Bloom Filter

In 1970, Burton Howard Bloom introduced a data structure called the *Bloom Filter* as a way to efficiently and compactly determine if an element belongs to a set [24]. This probabilistic data structure consists of a fixed-length vector of bits initialized with zeros. To check for membership of a specific input value, the Bloom Filter utilizes a set of independent hash functions, typically denoted as k , to select k index positions within the Bloom Filter. When an element is mapped to a particular index position by a hash function, the corresponding bit is updated to 1. However, due to the possibility of hash collisions, different elements may be mapped to the same index position. Consequently, when retrieving information about an element, there is a chance of encountering false positive matches in the query result, but false negatives are not possible.

A BF (BF) representing the set S having elements $S = \{a_1, a_2, \dots, a_n\}$ can be represented as:

$$BF(S) = \bigcup_{a \in S, h \in H} h(a) \quad (1)$$

where, $H = h_1, h_2, \dots, h_k$ is a set of k_{opt} independent hash functions such that each $h_i \in H : h\{a\} \rightarrow B\{1, \dots, l\}$, that is, the hash function takes each element present in the set as input and outputs a random number uniformly chosen in $\{1, \dots, l\}$.

Users can set the value of fp (i.e., hash collision) at the beginning of mapping data. Then, using the fp , the length of the Bloom Filter and optimal hash count can be calculated using Eq. (2) and Eq. (3) respectively:

$$l = \frac{m \times \log(fp)}{(\log 2)^2} \quad (2)$$

$$k = \frac{l}{m} \times \log 2 \quad (3)$$

where l is the length of the Bloom Filter and m is the total number of elements present in the dataset. The following subsection focuses on prominent research that employs bloom filter encoding.

1) *Privacy Preserving Data Encoding using Bloom Filter*: Since the invention of BF, researchers have used it in different research domains [25], [26]. However, Schnell *et al.* [27] was the first to use it for data encoding in Privacy-Preserving Record Linkage (PPRL). In their work, they showed how a BF could encode information about a person that was used to link several databases. Since then, it has been receiving widespread attention in several studies to encode information of different data types, such as categorical [28], string [27], [29], and numerical data [30].

Erlingsson *et al.* in [13] proposed a method, *RAPPOR*, to gather statistics from end-user client software in an anonymous manner while ensuring strong privacy protection by combining randomized response and Bloom filters. In essence, *RAPPOR* allows the analysis of the collective data from clients without the ability to examine individual data points. Through the innovative use of randomized response, *RAPPOR* enables efficient and valuable analysis of the collected data. Specifically, it facilitates the collection of statistics on client-side strings while guaranteeing strong privacy for each client and preventing the linkage of their reports. However, the encoding approach proposed in [13], using Bloom Filter (BF), was limited to handling single categorical (string) data. Their approach is inadequate for preserving the usefulness of data with larger scale/dimension and order information, such as images or time-series data.

Xue *et al.* [14] presented a new technique using Bloom filters for maintaining privacy in sequential data matching and distance-based machine learning algorithms such as KNN, SVM, and Random Forest. The method achieves privacy preservation through a two-step process. First, a Bloom filter is constructed by mapping a vector of numerical values from the sequential data. Second, privacy is enhanced by introducing differential privacy-based noise to the output of the Bloom filters, utilizing a Laplace-like mechanism. The authors claim that their work represents a modest advancement in privacy-preserving machine learning (PPML) by utilizing perturbed Bloom filters instead of cryptographic methods like homomorphic encryptions.

While the data encoding method proposed in [14] presented a novel approach utilizing BF-encoding and its applicability in traditional machine learning (ML) algorithms, we have identified several limitations in their research. Firstly, their proof of the privacy definition (ϵ -DP guarantee) is flawed. Secondly, they failed to acknowledge the limitations of the DH technique. Niedermeyer *et al.* [20] demonstrated the potential for initiating a cryptanalysis attack on the Bloom Filter through pattern-mining, exploiting the vulnerabilities of the DH technique. Thirdly, they failed to assess the performance of the BF-encoded dataset in DL learning algorithms and solely focused on classification tasks using traditional ML algorithms. On the other hand, it is crucial to determine whether BF-encoded data can be considered an innovative option for preserving privacy in more complex tasks like time-series forecasting.

Despite numerous significant studies conducted on BF-encoding and its potential in safeguarding privacy during data analysis, the current state-of-the-art BF-encoding method possesses certain constraints, leading to various unresolved

research inquiries. These inquiries include determining the feasibility of encoding color images and time-series data using BF, as well as assessing the usability of BF-encoded datasets in deep learning algorithms. Additionally, there is a need to investigate whether it is possible to perform forecasting with BF-encoded time-series data and to explore the privacy implications of machine learning models trained on these encoded datasets in the face of membership inference attacks (MIA). To overcome these limitations and address the aforementioned research questions, our study aims to enhance the existing encoding technique and provide solutions.

B. Differential Privacy

Differential Privacy establishes a precise benchmark for guaranteeing that the results of an algorithm do not expose the personal information of individuals present in its input data. A common approach to achieve differential privacy involves assessing a function based on the input data and subsequently introducing a minor degree of random noise to the outcome prior to its disclosure [31].

DEFINITION 1: (ϵ -Differential Privacy [10]).

A randomized function $F : D^n \rightarrow D^n$ is ϵ -differentially private, if for all neighboring profiles $x, x' \in D^n$ and for all $t \in D^n$:

$$\Pr[F(x) = t] \leq \exp(\epsilon) \cdot \Pr[F(x') = t]. \quad (4)$$

C. Local Differential Privacy

In Local differential privacy (LDP), the data owners apply randomization (or noise) to the input data before sharing it with the third-party service provider. LDP provides a better privacy notion than central DP due to the increased noise levels and the non-necessity of a trusted curator [32].

DEFINITION 2: (ϵ -Local Differential Privacy [33]).

A randomized algorithm A satisfies ϵ -local differential privacy if and only if for any two input values x_1 and x_2 and any output $O \subseteq \text{Range}(A)$ and for $\epsilon \geq 0$, we have

$$\Pr[A(x_1) \in O] \leq \exp(\epsilon) \cdot \Pr[A(x_2) \in O]. \quad (5)$$

where $\Pr[\cdot]$ represents probabilities and ϵ is the privacy budget.

1) *Laplace Mechanism*: Suppose $\text{Laplace}(\sigma)$ is the Laplace distribution, whose probability density function is $f(x) = \frac{1}{2\sigma} \exp(-\frac{|x-\mu|}{\sigma})$, where μ is a location parameter and $\sigma > 0$ is referred as scale parameter. Dwork *et al.* proved that, for a given query function F and a database D , a randomized mechanism M_r that returns $F(D) + \gamma$ as an answer where γ is selected i.i.d from $\text{Laplace}(\frac{\Delta F}{\epsilon})$, gives ϵ -differential privacy [34]. We utilized the Laplace Mechanism to add noise to BF such that the BFs are differentially private.

Considering the LDP model [13], [33], we assume that the aggregator already knows the identities of the users, but not their private data. Suppose, there are N users, and u_i ($1 \leq i \leq N$) denote the i -th user. Each user u_i 's private data is represented by a tuple t_i , which contains d features $F_1, F_2, \dots, F_d$. These features are first encoded in the Bloom Filter (BF_i). After that to ensure privacy, LDP noise is perturbed to each BF_i using the randomized

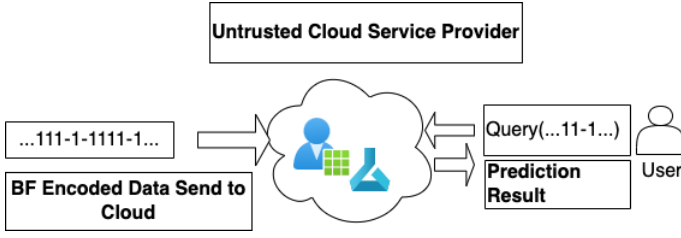


Fig. 1. Cloud-Enabled Data Analysis.

perturbation function M_r . Then, the user sends the perturbed data $M_r(BF_i)$ to the aggregator instead of the original data record t_i . Given a privacy parameter $\epsilon > 0$ that controls the privacy-utility tradeoff, we proved that M_r satisfies ϵ -local differential privacy (ϵ -LDP) (Theorem 1).

III. THREAT MODEL

This research considers a typical cloud-enabled network, where the data aggregator collects the encoded data from users and sends it to the cloud for storage and processing. The cloud provider hosts different data analysis frameworks for DL analysis. The goal is to maximize the utility of the perturbed encoded data at the same time preserve the privacy of the users. The cloud provider is considered an honest-but-curious [35] adversary, who may not alter the encoded data but will try to retrieve as much information as possible for the sake of its business. We also assume that there is a chance of public availability of the plain text data, which some other service provider may publish. Based on these settings, we consider that an adversary may initiate a pattern-mining-based attack on the encoded data to decrypt it and a membership inference attack on the DL model to reveal the membership information of the training data.

A. Pattern-Based Attack on the BF-Encoded Data

The BF-encoded datasets are vulnerable to pattern-based attacks due to the weakness of the hashing technique (i.e., DH) used to map the data in the Bloom Filter. Schnell and Borgs [21] proposed Random Hashing (RH) instead of Double Hashing (DH) to prevent pattern-based attacks on Bloom Filter. As the DH (Eq. (6)) is based on the linear combination and addition of two integer hash values, it generates a repeatable pattern [36]. In RH (Eq. (7)), a specific random seed is initialized based on the element being hashed to generate the k_{opt} random number using a pseudo-random number generator (PRNG). The generated random numbers are then used to set the bit position of the Bloom Filter to 1. A PRNG, if initialized to a specific seed value α , will always generate the same sequence of pseudo-random numbers [36].

$$B_i(\alpha) = (h_1(\alpha) + i * h_2(\alpha)) \mod l \quad (6)$$

$$B_i(\alpha) = \text{randint}(0, l - 1), \text{ where } 1 \leq i \leq k_{opt} \quad (7)$$

Christen *et al.* [36] shows that according to modulo arithmetic Eq. (6) of DH can be written as follows:

$$B_i(\alpha) = [(h_1(\alpha) \mod l) + (i * h_2(\alpha)) \mod l] \mod l \quad (8)$$

where, h_1 and h_2 are two independent cryptographic hash functions, l is the length of the BF and α is the data to be mapped in the BF. Here $0 \leq i \leq k_{opt}$, where k_{opt} is the total number of hash functions. Let, $p = [h_1(\alpha) \mod l]$ and $q = [i * h_2(\alpha) \mod l]$, then $0 \leq p < l$ and $0 \leq q < l$.

Therefore, for a finite BF length, l , there will be only l^2 possible value pairs of (p, q) to map the data. So the dataset will be mapped by a considerably small number of feasible patterns, and adversaries may take advantage of this weakness [20].

Countermeasure: We use the Random Hashing technique as a countermeasure, which maps the data randomly and can prevent a pattern-based attack. In RH, k_{opt} random numbers are drawn for every possible member of the neighbor-added dataset. For example, let's consider the length of the BF as 1000. The number of neighbors equals two for each data point, and $k_{opt} = 10$ hash functions, then approximately 10^{21} different combinations are possible. Whereas for the DH, it will be only 10^6 , which is a minimal number corresponding to a cryptanalysis attack. In addition, to prevent the graph-matching-based attack, we consider full-length feature vectors of the data as suggested in [37].

B. Membership Inference Attack

ML models are vulnerable to membership inference attacks, where adversaries may attack the model and identify the participants present in the dataset. Shokri *et al.* [18] mentioned that ML models, which are differentially private, limit the success probability of membership inference attacks (MIA) based solely on the model.

Countermeasure: As a countermeasure, we ensure ϵ -DP of the encoded dataset by adding Laplace Noise to them. We test the privacy of the ML model against MIA by using the attack model of TensorFlow Privacy [22].

IV. SYSTEM OVERVIEW

In accordance with our Threat Model outlined in Section III, Fig. 1 illustrates the cloud-enabled data processing framework, where the data aggregator publishes the perturbed BF-encoded data to the untrusted cloud. The cloud provider hosts and trains a deep learning model using this encoded data, which will subsequently be used for data inference. It's worth noting that encryption of the responses can be implemented using established encryption algorithms as mentioned in [38], [39]. However, it's important to clarify that our current research does not delve into the encryption aspect. Notably, some recent studies have utilized Label-DP [40], [41], where they assume that the feature sets are publicly available but consider the class labels to be sensitive. In contrast, our approach treats the feature vector as sensitive. In future work, we intend to explore the application of differential privacy to both feature sets and labels.

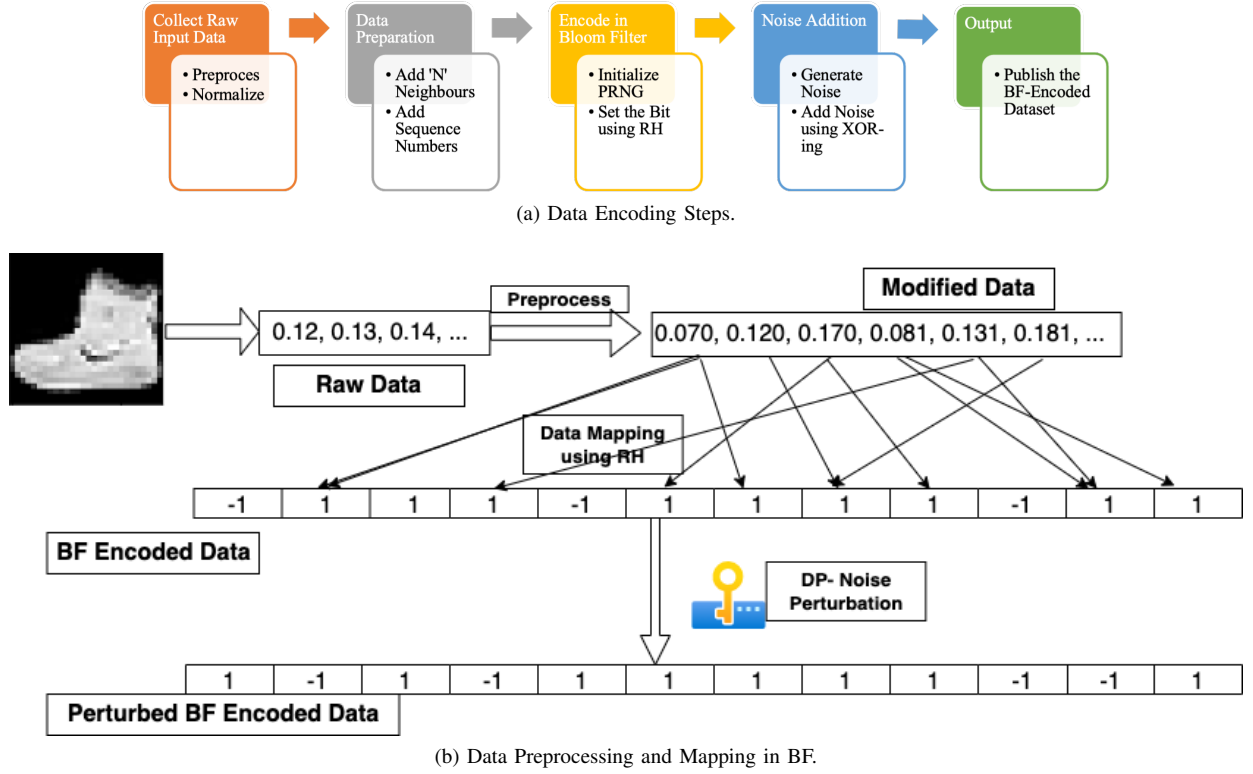


Fig. 2. Data Encoding.

Next, we will describe our proposed BF-encoding technique, which is designed to encode more intricate datasets, as well as the employment of a deep learning (DL) model for data analysis. Specifically, our research applies BF-encoding to color images, facial images, and time-series datasets. Subsequently, we employ CNN-based classification and forecasting models to detect objects and forecast future values.

A. Data Encoding

Our proposed approach for Bloom Filter (BF) encoding involves three main steps: Data Preprocessing, Data Mapping in BF, and Noise Addition. Fig. 2(a) and (b) provide a visual representation of these encoding steps. The image datasets are first pre-processed and then mapped into the BF. For time-series datasets, we initially divide them into training and test sets. These sets are further divided into observed and forecast data. For instance, if the observed data consists of values at time-stamps i , $i+1$, and $i+2$, then the corresponding forecast data would contain values at time-stamp $i+3$. Next, we encode the observed data points from the training and test datasets into the BF. These encoded datasets are then passed to the next step, which involves adding random noise. Algorithm 1 shows the pseudo-code of the Differentially Private Bloom Filter Encoding technique.

1) *Data Preprocessing*: The initial step involves normalizing the raw datasets. Subsequently, for each feature vector, an additional $2n$ neighboring data points are included that are in close proximity to each feature value of a training sample. The purpose is to enhance the likelihood of similarity between two BFs that contain similar information sets. The updated

set comprises the original feature value along with its $2n$ neighbors. In general, for a given feature x , the *neighbor Set* of x is generated within the range $[x - \frac{\delta}{2}, x + \frac{\delta}{2}]$, with intervals of size $\Delta = \frac{\delta}{2n}$. Here, δ represents the difference between the maximum and minimum values of the normalized data points.

To illustrate, let's take a set of numerical feature vectors ranging from 16 to 20, and we aim to hash-map $x_1 = 16$, $x_2 = 17$, and $x_3 = 19$ with a parameter $n = 2$. In this scenario, the value of δ is 4, and Δ is 1. Consequently, the newly generated sets for 16, 17, and 19 will be denoted as $L_1 = [14, 15, 16, 17, 18]$, $L_2 = [15, 16, 17, 18, 19]$, and $L_3 = [17, 18, 19, 20, 21]$. The number of common elements between L_1 and L_2 can be calculated as $|L_1 \cap L_2| = [15, 16, 17, 18] = 4$, and between L_1 and L_3 as $|L_1 \cap L_3| = [17, 18] = 2$ elements. Hence, based on the set similarity metric (e.g., Dice-coefficient similarity, Please refer to Appendix D for details), the similarity between x_1 and x_2 is computed as $\text{sim}(x_1, x_2) = \frac{24}{5+5} = 0.8$. Similarly, the similarity between x_1 and x_3 is calculated as $\text{sim}(x_1, x_3) = \frac{22}{5+5} = 0.4$. Based on these similarity values, it is determined that between x_2 and x_3 , x_2 is more similar to x_1 than x_3 [30].

Subsequently, the index position is appended to each data point to ensure their separation. This step is included to preserve the sequential order of the data that comes in multiple times. Without this adjustment, hashing would result in mapping only once in the BF of those values that appear several times in the original set, resulting in a loss of information. Importantly, the index position assigned to the generated neighbor for each feature vector will be the same as the index of that feature vector. For instance, consider an example where

Algorithm 1 Differentially Private Bloom Filter Encoding.

```

1: inputs: raw dataset:  $D^{N \times d}$ , neighbor count:  $n$ , neighbor
   distance:  $\Delta$ , hash no.:  $k$ , BF-length:  $l$ , privacy parameter:  $\epsilon$ .

2: output: perturbed Bloom Filter:  $BF'^{N \times l}$ 
3: initialize: initialize the Bloom Filter:  $BF^{N \times l} \leftarrow 0$ 
4: for each user  $i \in N$  do
5:   for each features  $j \in d$  do
6:     Generate  $2 \cdot n$  neighbors for each features those are
        $\Delta$  distance apart
7:     Add index number to each neighbor based on the
       original feature index
8:     Map each member into  $k$  random position of the  $BF$ 
       and set that bit position as 1.
9:   end for
10: end for
11: initialize: initialize an array:  $noise^{N \times l} \leftarrow \phi$ 
12:  $\zeta = \frac{1}{1 + \exp(\frac{\epsilon}{2 \cdot k})}$ 
13: for  $i = 1, 2, \dots, N$  do
14:   Generate the random Laplace noise  $LAP^{1 \times l}$  with
       mean = 0 and scale = 1
15:   for  $j = 1, 2, \dots, l$  do
16:      $p = LAP[j]$  {Laplace noise is the random variable}

17:     if  $p \geq \zeta$  or  $p \leq -\zeta$  then
18:        $noise[i][j] = 1$ 
19:     else
20:        $noise[i][j] = 0$ 
21:     end if
22:      $BF'[i][j] = BF[i][j] \oplus noise[i][j]$ 
23:   end for
24: end for
25: Return the perturbed Bloom Filter,  $BF'^{N \times l}$ 

```

the original training set comprises feature vectors [16, 16, 20]. After the preprocessing step (i.e., neighbor addition and appending the index position), the modified set would contain the following values: ["1401", "1501", "1601", "1701", "1801", "1402", "1502", "1602", "1702", "1802", "1803", "1903", "2003", "2103", "2203"].

2) *Data Mapping in BF:* Following the pre-processing step, the modified dataset is mapped into the Bloom Filter (BF) using Random Hashing. First, a Pseudorandom Number Generator (PRNG) is initialized with the seed value. Next, for each data point, the PRNG generates k random indices and those index positions of the BF are set to 1. We adopt the Random Hashing (RH) mechanism to map all the feature vectors into the Bloom Filter, as an alternative to the previous Double Hashing (DH) technique. The aim is to reduce vulnerability to pattern-based attacks [20] on the BF. Furthermore, we encode all the feature vectors to mitigate graph-matching attacks [37] on the BF. For more detailed information on the graph-matching attack on BF, please refer to Appendix A.

3) *Noise Addition:* Next, we add Random Noise to the encoded dataset to ensure ϵ -differential privacy. First, the Random Noise is generated from the Laplace Distribution,

$LAP(0, 1)$. Then this random variable is converted into binary values by using a threshold parameter ζ . Next, the binarized noise data is XOR-ed with the corresponding bit position of the encoded dataset. Due to XOR-ing, the output of the BF changed only when the noise was 1. The perturbed-BF (BF') output can be represented with the following probabilities, where BF represents the original BF encoded data:

$$BF'_i = \begin{cases} BF_i, & \text{with probability } 1 - p, \text{ (i.e., noise is 0)} \\ \neg(BF_i), & \text{with probability } p, \text{ (i.e., noise is 1)} \end{cases} \quad (9)$$

The noise-added encoded datasets are published (i.e., sent to the cloud) for future data analysis (i.e., training the model or query processing).

Adding binarized random Laplace noise ensures ϵ -LDP privacy guarantee to the encoded data. To derive the relation between p and ϵ we propose the Theorem1:

THEOREM 1: The addition of Laplace distribution noise thresholded by ζ -value to a Bloom Filter satisfies ϵ -differential privacy where $\epsilon = 2 \cdot k \cdot \ln \left(\frac{1-p}{p} \right)$.

Please see the Appendix C for Proof of the Theorem 1

a) *Issues in the Proof of ϵ -DP in [14]:* We identify an issue in deriving the proof of ϵ -DP presented in the previous research [14]. The authors mentioned, "Assume that $P = f$ is the probability of 1s and $P = 1 - f$ is the probability of 0s in the output of Bloom filters. The addition of Laplace noise will affect the distributions of 1s and 0s in the output of Bloom filters, which depends on the noise budget (e.g., p-value)". To quantify the noise impact of p-value on the output of Bloom filters, they develop the theorem mentioning ϵ value as:

$$\epsilon = 2 \cdot f \cdot n \cdot \ln \left(\frac{p - p \cdot f}{1 - p + p \cdot f} \right) \quad (10)$$

And in the proof, the probability of changing the bit of the perturbed Bloom Filter is represented by the following two equations:

$$P(bf'_i = 1 \mid bf_i = 1) = p - p \cdot f, \quad (11)$$

$$P(bf'_i = 1 \mid bf_i = 0) = 1 - p + p \cdot f \quad (12)$$

As they XOR-ed the noise with the output of the original Bloom Filter, we can derive the Truth Table I. The estimated probabilities are mentioned in the bracket.

TABLE I
THE OUTPUT OF PERTURBED BF BASED ON THE XOR-ING AND PROBABILITY ESTIMATION DESCRIBED IN [14]

BF	Noise	BF'
0 (1-f)	0 (1-p)	0
0 (1-f)	1 (p)	1
1 (f)	0 (1-p)	1
1 (f)	1 (p)	0

Based on the Truth Table I, equations 11 and 12 should be expressed as follows:

$$P(bf'_i = 1 \mid bf_i = 1) = f - p \cdot f, \quad (13)$$

$$P(bf'_i = 1 \mid bf_i = 0) = p - p \cdot f \quad (14)$$

Hence the value of ϵ will be different from the given one.

Another concern is, if we consider $f = 0.5$ and $p = 0.3$, $n = 100$, then from 10 it gives $\epsilon = -173.46$, which is not justified.

B. The DL Model for BF-Encoded Data Analysis

We considered that the cloud service provider that the cloud service provider might implement a Deep Learning model based on Convolutional Neural Networks (CNNs) [42]. This model would be utilized to classify and predict outcomes using the perturbed BF-encoded dataset. The classification model consisted of two blocks of CNN layers followed by two blocks of Dense layers. Each CNN block was comprised of a CNN layer, followed by Activation (Rectified Linear Unit (ReLU)), Max Pooling, and Dropout Layer. Each Dense block except the last layer consisted of the Dense Layer, followed by the Activation (ReLU) and Dropout Layer. The final block had a Dense layer and a Softmax Activation. We used Adam as the optimizer and Cross-entropy as the loss function for this model. The Forecasting Model consisted of one block of CNN, followed by two blocks of the Dense layer. The CNN block contained the 1D-CNN (i.e., Temporal CNN) layer, followed by Activation (ReLU) and 1D-Max Pooling Layer. Here we used the Stochastic Gradient Descent (SGD) algorithm as the optimizer and Mean Squared Error (MSE) as the loss function. We used *EarlyStopping* and *LearningRateScheduler* strategies as *Callbacks* while training the CNN model to reduce overfitting as it increases the risk of membership inference attacks. We chose to train shallow models as training deeper models may be detrimental to generalization when learning with privacy[43]. In order to enhance the effectiveness of the BF-encoded data within the Deep Learning model, we modify the unset bit position of the BF (0 values) as -1, taking inspiration from the binarization principle found in Binary Neural Networks [44]. We have provided a detailed description of CNN architecture in Appendix E.

V. EXPERIMENTAL EVALUATION

In this section, we present and discuss the results of the experimental study of our proposed method.

A. Experimental Setting

a) **Dataset:** We used five public datasets in this research. Among them were the Fashion-MNIST [45] and CIFAR-10 [46] which contain images of different objects. The third and fourth were the Facial Image Dataset (Labeled Faces in the Wild (LFW) [47] and Gender (Face) Identification Dataset [48]), and the fifth was household-electricity-usage dataset [49], i.e., the time-series dataset.

The **Fashion-MNIST** dataset contains images of 10 different types of clothing: t-shirts, trousers, pullover, dress, coat, sandals, shirt, sneaker, bag, and ankle boots. The training set has 60,000, and the test set has 10,000 samples of 28 x 28 grayscale images. We randomly sampled the dataset for this

experiment and selected 10000 samples for training and 1000 samples for the test dataset. Considering a hash collision rate of 0.5 and the total number of neighbors as being 20, we calculated the BF length and hash count as 50000 and 2, respectively.

The **CIFAR-10** dataset contains 32 x 32 color images of 10 different objects: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. The training set contains 50000 samples, and the test set contains 10000 samples. We randomly sampled the CIFAR-10 dataset and selected 10000 samples for training and 1000 samples for the test dataset. Considering a hash collision rate of 0.5 and the total number of neighbors as 10, we calculated the BF length and hash count as 36100 x 3 (190 x 190 x 3) and 2, respectively.

The **Labeled Faces in the Wild (LFW) Dataset** The Labeled Faces in the Wild (LFW) dataset contains more than 13,000 images of faces collected from the web. Each face has been labeled with the name of the person pictured. We used the pre-processed excerpt of this dataset from the scikit-learn library [50]. The used dataset contains 100 images per person resulting in 5 class samples. The training set contains 855 samples, and the test set contains 285 samples. For the facial image dataset, we considered 20 neighbors and a hash collision rate of 0.7. With these values, the measured BF length was 80000, and the hash count was 2.

The **Gender (Face) Identification Dataset** This dataset contains color facial images of males and females, having around 2300 samples. The author [48] mentioned that the Face region is cropped by applying face detection using *cvlib* [51] on the images gathered from Google Images. We used 80% of the data as training and 20% as test data. The algorithm identifies the male and female images from the data. We considered 10 neighbors and a hash collision rate of 0.64. With these values, the measured BF length was 80,000, and the hash count was 2.

The **Electricity-Usage** dataset was collected by the Pecan Street Inc's Dataport site [49], [52] and contained circuit-level electricity usage data starting from January 01, 2018, 00:00:00 to December 31, 2018, 23:45:00. The meter reading was collected at 15 minutes intervals. The data spanned from negative to positive values. The positive values referred to the normal house energy consumption from the power grid, and the negative values referred to the energy contribution from the house to the power grid. We chose a subset of 25 homes' data for a year. There are approximately 35000 usage records for each user. We used the three months of data as a train and five days of data as a test set for each user. The forecasting algorithm considered two hours of encoded data (i.e., eight consecutive usage records) as observation and, based on that, forecasted the electricity usage for the next quarter. Considering the hash collision rate of 0.05 and the total number of neighbors, 10, we calculate the length of the BF and hash count as 1047 and 2, respectively.

b) **Performance Metric:** The metric *Accuracy* was used to quantify the performance of the object detection and facial recognition model, and the *RMSE* and *MAE* were used to measure the forecasting error. The high classification accuracy and low forecasting error validate the utility of the encoded

TABLE II
CLASSIFICATION ACCURACY

Dataset	Accuracy (Plain)	Accuracy (BF-Encoded)
Fashion-MNIST	91%	86.2%
CIFAR-5	93%	83%
LFW	90%	83.16%
Gender	96%	88%

TABLE III
FORECASTING ERROR

Dataset	RMSE (Plain)	RMSE (Encoded)	MAE (Plain)	MAE (Encoded)
Electricity Usage	0.101	0.139	0.056	0.087

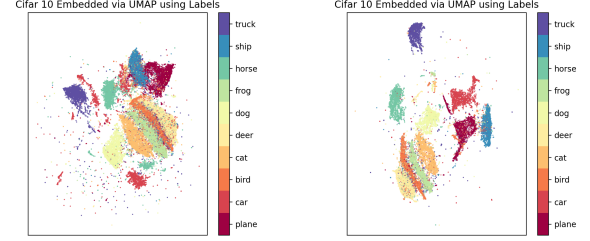
data.

B. Utility Analysis of the BF-Encoded Dataset in DL Model

We utilized the proposed BF-encoding technique on the aforementioned five datasets, incorporating the encoded data into our DL model. Tables II and III present the most favorable results achieved using these encoded datasets. Here, we demonstrate the classification accuracy (Table II) and forecasting error (Table III) for both the original and encoded datasets within the deep learning model. For the Fashion-MNIST datasets, the classification accuracy stands at 91% for the unencoded data and 86.2% for the encoded data. In the case of the CIFAR-10 dataset, the classification accuracy exceeds 73% for the plain dataset and drops to 60% for the encoded dataset. Moving on to the Labeled Faces in the Wild (LFW) dataset, we observe a classification accuracy of 90% for the unencoded data and 83.2% for the encoded data. As for the Gender (Face) Identification dataset, the classification accuracy reaches 96% for the plain data and 88% for the encoded data. Finally, in the context of the Household Electricity Usage dataset, the forecasting error for the plain data is recorded as RMSE 0.101, whereas for the encoded data, it registers as RMSE 0.139.

1) *A Deeper Insight into the Performance of the CIFAR-10 Dataset:* As the encoded CIFAR-10 dataset's classification accuracy was lower compared to other datasets, we conducted a deep investigation to understand the data distribution of the plain and encoded datasets. First, we generated the 2D embedding UMAP [54] plot to confirm whether the encoded datasets had samples from ten different classes or not. Fig. 3(a) and (b) are generated from the plain and encoded data respectively. The figures show that in both embedding plots, there are samples from ten different classes but they are overlapping, and there are some outliers present in both datasets.

In addition to this, Sarker *et al.* in [53] showed that there are some noisy labels present in the CIFAR-10 test dataset. Their research demonstrated that images of *Plane* and *Dogs* could be mislabeled into *Ship*, *Horse*, *Cat* and *Bird* classes Fig. 3(c). Therefore, such classes include noisy samples resulting in the lower classification accuracy of the encoded dataset.

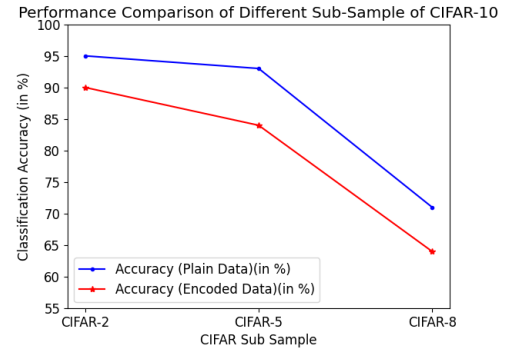


(a) CIFAR-10 UMAP Plot(plain).

(b) CIFAR-10 UMAP Plot (encoded).



(c) Noisy labels present in the CIFAR-10 Test Dataset [53].

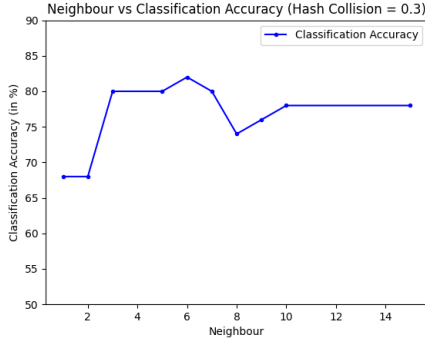


(d) Performance Comparison of Different Sub-Samples of CIFAR-10 Data.

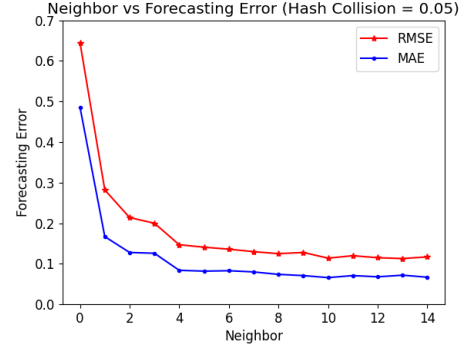
Fig. 3. CIFAR-10 Data Analysis.

Hence we randomly sub-sampled the dataset, removing the samples from those classes with noisy labels and recording the classification accuracy. Initially, we considered samples from only two classes (CIFAR-2) (airplane and automobile). Later we considered samples from five (CIFAR-5) (airplane, automobile, deer, frog, truck) and eight classes (CIFAR-8). The sub-sample CIFAR-2 and CIFAR-5 do not have any noisy labels, but the CIFAR-8 do. Fig. 3 (d) shows the classification accuracy of plain and encoded sub-sample datasets of CIFAR-10. The classification accuracy of the encoded CIFAR-2 and CIFAR-5 datasets are very close to the plain samples, but this drops for CIFAR-8 data. Table II list the classification accuracy of the plain and encoded CIFAR-5 dataset.

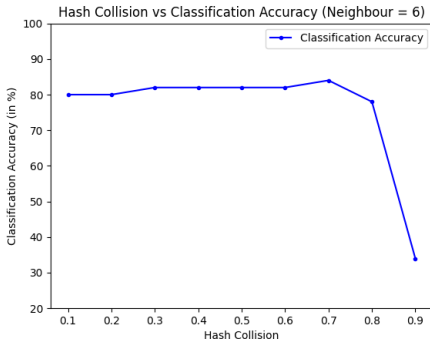
Furthermore, Papernot *et al.* demonstrated in their paper [43] that in privacy analysis, when considering shallow DL architecture, the accuracy of the plain CIFAR-10 dataset was 76.6%. However, when considering DP, the accuracy decreased to 61.6%, which supports our findings.



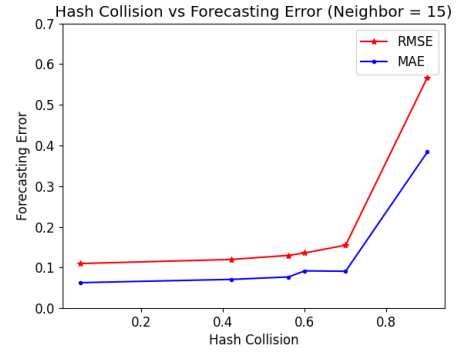
(a) Impact of neighbor on Classification Accuracy.



(b) Impact of neighbor Addition on Forecasting Error.



(c) Impact of Hash Collision on Classification Accuracy.



(d) Impact of Hash Collision on Forecasting Error.

Fig. 4. Impact of Different Encoding Parameters

VI. IMPACT OF DIFFERENT ENCODING PARAMETERS

We tested the impact of different encoding parameters. This section will discuss those findings. Here, we considered the encoded color image (i.e., CIFAR-5) datasets for the following experiments.

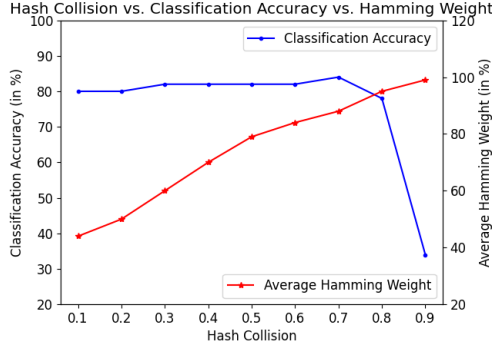
a) Neighbor Size: Fig 4(a) shows the impact of adding neighbors to CIFAR-5 dataset for a constant hash-collision rate (0.3). The classification accuracy increased up to 84% when the neighbor size was six. Then it started to drop. After that, some randomness appeared in the classification accuracy for this dataset. Fig. 4(b) shows the impact of increasing neighbor size on the forecasting error. Here the hash collision rate was kept constant (0.05). This experimental result shows that increasing the number of neighbors lowers the forecasting error for a constant hash collision rate. The forecasting error keeps decreasing until the neighbor size is 10. After that, it shows some randomness. In opposition with [14] here, we kept the neighbor distance Δ constant. As we did not apply any compression algorithm (i.e., SVD), there was no change in the initial data distribution. However, this parameter is data-dependent.

b) Impact of Hash Collision rate: Fig. 4(c) shows the impact of the hash collision rate on the classification accuracy of the encoded CIFAR-5 dataset. We changed the hash collision rate for the neighbor number $n = 6$. The classification accuracy dropped after the hash collision became > 0.7 . Fig. 4(d) shows how the forecasting error changes with the change of hash collision rate. Here we varied the hash collision rate

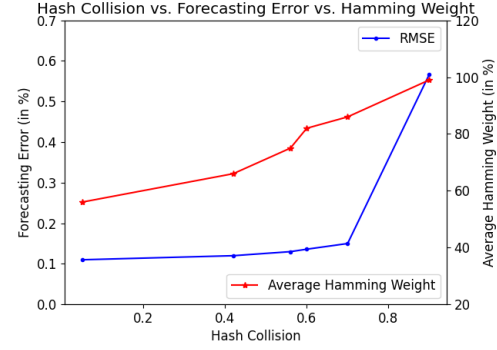
for a constant neighbor number of $n = 15$ and observed a sharp spike in the error after the hash collision rate was > 0.7 .

c) Impact of Hamming-Weight: Authors in [55] mentioned that having 50% of bits set to 1 (i.e., Hamming Weight) in a Bloom Filter should be optimal in terms of a hash collision (or false positive (fp) rate). Borgs *et al.* in [56] showed that for privacy-preserving record linkage, optimal linkage quality can be achieved when the Hamming Weight (total number of bits set to 1 in the bloom filter) is up to 85%, but after that, precision drops sharply when the Hamming Weight reaches $> 90\%$. In our experiment, we calculated the average Hamming Weight for the training dataset. We found that with the increase of hash collisions, Hamming Weight also increased, and classification and forecasting performance also improved (Fig.5(a) and 5(b)). However, the performance drop just after the value of the Hamming weight was greater than 70%.

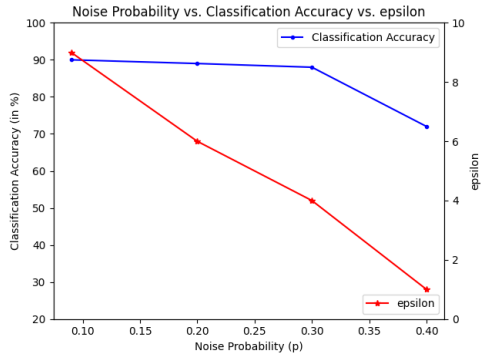
d) Impact of Privacy Budget (ϵ): We perturbed binarized Laplace noise on the encoded dataset to ensure a differential privacy guarantee. Fig. 5(c) and 5(d) show the noise probability (p) vs. privacy budget (ϵ) vs. classification accuracy and forecasting error, respectively. The classification accuracy dropped with the increase in privacy budget (ϵ). For the electricity dataset, the forecasting error increased with the increase of it.



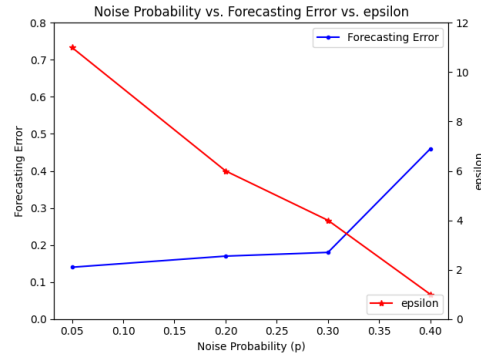
(a) Impact of Hamming Weight on Classification Accuracy.



(b) Impact of Hamming Weight on Forecasting Error.



(c) Impact of Privacy Budget on Classification Accuracy.



(d) Impact of Privacy Budget on Forecasting Error.

Fig. 5. Impact of Different Encoding Parameters

VII. WHY DL ALGORITHM?

A. Utility Analysis of the DL Model

a) *Classification Model*: Traditional ML algorithms like KNN and SVM do not always perform well on all datasets. For example, using these algorithms, we got very low classification accuracy for CIFAR-10 and Facial image datasets compared to the CNN-based deep learning algorithm. The reason is that CNN-based models are more potent in extracting features from the images than the traditional ML model, which gives better classification accuracy. Similarly, for the encoded datasets, our CNN-based model performs better than the state-of-the-art ML algorithms. We tested using the Network-in-Network (NiN) [57] model, and this algorithm did not show satisfactory results on the encoded datasets. Fig. 6 (a) shows the classification accuracy of KNN, SVM, NiN, and our CNN-based model (BloomCNN) on plain and encoded CIFAR-5 datasets. We observed similar performance for other datasets as well.

b) *Forecasting Algorithm*: We used the statistical and DL-based forecasting models on plain electricity data to forecast future value. We used Auto-Regressive Integrated Moving Average (ARIMA) and Exponential Weighted Moving Average (EWMA) algorithms among traditional time-series forecasting algorithms. In addition, we used the temporal (one-dimensional) CNN-based model and Long Short Term Memory (LSTM) network. For the ARIMA and EWMA, we selected value eight as the lag observation and span parameter,

respectively. Fig. 6 (b) shows that the 1D-CNN-based deep learning model could forecast electricity usage with the lowest error (0.101) among different forecasting algorithms.

We did the same experiment using the BF-encoded dataset, and only our model could forecast with a meager error rate (0.114). The forecasting errors for EWMA and LSTM were relatively high (0.635 and 0.313, respectively). The ARIMA model could not operate using the encoded datasets as these datasets contained only a series of +1s and -1s; the statistical algorithms were inefficient in retrieving enough information from there. In contrast, during training, the DL algorithm used the backpropagation and SGD optimization algorithms, which helped update the model parameters and forecast with lower error. For the LSTM network, due to binary data, there was some risk of vanishing gradient problem, which caused poor performance compared to the CNN-based model [58].

B. Privacy Analysis of the DL Model

TABLE IV
ATTACKER ACCURACY IN MIA.

Dataset	Attacker Accuracy
Fashion-MNIST	51%
CIFAR-5	50%
LFW	48%
Gender Face	51%
Electricity	49%

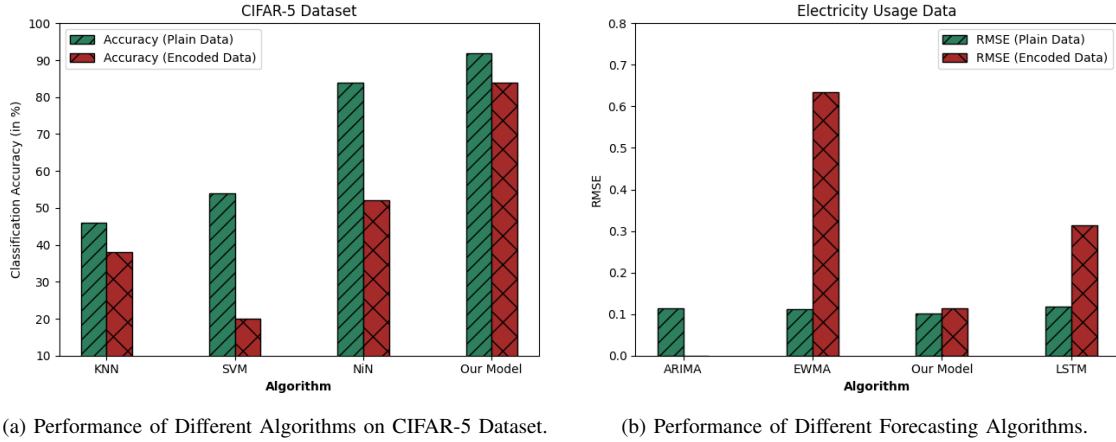


Fig. 6. Performance Comparison of Different ML/DL Model on Plain and BF-Encoded Data.

To ensure the differential privacy guarantee, we perturbed the encoded data with random noise, and Theorem 1 proves that it provides an epsilon-DP guarantee. Due to the post-processing properties of differential privacy, once differential privacy is applied to the dataset, all further processing (including joint computations over outputs of other analyses) does not weaken differential privacy guarantees [10]. However, the ML/DL models are vulnerable to MIA [23], where an attacker with sufficient knowledge can extract membership information of the training dataset from the trained model parameters. To estimate the risk of MIA, we assessed our model using the TensorFlow-Privacy's Membership Inference Attack (MIA) module. We simulate the attack on the model and measure the attacker's accuracy. We assumed the attacker somehow succeeded in accessing some trained model parameters and auxiliary datasets. We considered both cases where there is a chance that the attacker gains access to raw and encoded data and tries to reveal the membership information by attacking the model. We considered a black-box attack. Table. IV shows the maximum attacker accuracy from the membership information attack. Here we evaluate the privacy of the DL model while training using the BF-encoded dataset. The attacker accuracy in revealing membership information for Fashion-MNIST was 0.51; for CIFAR-5, it was 0.5; for LFW, 0.48; for GenderFace, 0.51 and for Electricity data, it was 0.49. It is to be noted that the attacker's accuracy (ROC/AUC) value close to 0.5 signifies the attacker's inability to identify membership information from the model. Therefore, the results demonstrate that the attacker could only make random guesses for all BF-encoded datasets as the AUC was around 50%.

VIII. COMPARISON WITH THE STATE-OF-THE-ART BF-ENCODING

This study aims to address the limitations identified in a previous work [14], which extended the RAPPOR technique [13]. Our experiments revealed that the use of the encoding methods proposed in [13] and [14] resulted in significantly reduced accuracy when applied to Deep Learning-based classification algorithms.

TABLE V
IMPACT OF DIFFERENT ENCODING TECHNIQUES ON DATA UTILITY (DATASET: MNIST)

BF-Encoding	Data Utility (Classification Accuracy)
RAPPOR [13]	14%
SOTA [14]	63%
Our Approach	93.8 %

TABLE VI
IMPACT OF DIFFERENT DP-NOISE ON DATA UTILITY (DATASET: FASHION-MNIST)

DP-Noise	Data Utility (Classification Accuracy)
Raw Dataset (No Noise Added)	91%
Randomized Response Noise	76%
Binarized Laplace Noise	86 %

The encoding method proposed in [13] utilized Bloom Filter (BF) but was restricted to handling single categorical (string) data. It involved mapping the string data using multiple hash functions and applying the Randomized Response technique for ensuring differential privacy (DP). However, this approach was inadequate for preserving the usefulness of data with larger scale/dimension and order information, such as images or time-series data. In the case of images, where the same pixel value repeats in different index positions, the encoded dataset failed to maintain the order of similar pixel values appearing in the image. This limitation is also applied to time-series datasets. To overcome this issue with the BF-encoding technique from [13], we included the index position of each pixel value. Additionally, to enhance the security against cryptanalysis attacks [20] on the BF, we employed Random Hashing (RH) instead of Double Hashing (DH). Furthermore, we modified the data representation of the BF to improve the utility of the encoded dataset in DL algorithms, enabling the use of the BF-encoded dataset in time-series forecasting using a CNN-based model. Through experimental results Fig. 6 (b),

we demonstrated that state-of-the-art forecasting algorithms were unable to extract information (i.e., make accurate forecasts) from the encoded data.

A. The Impact of Different Data Encoding Techniques on Data Utility in DL Algorithm

We compared the utility of our BF-encoded data with the BF-encoded data proposed in RAPPOR [13] and previous research [14] in the BloomCNN model. First, we randomly sampled the MNIST dataset [15] having 6000 training and 1000 test samples. We chose this dataset, as it was used in [14]. After that, the data encoded with three different BF-encoding approaches are used in the BloomCNN model for digit identification. Our experimental results show that the classification accuracy of BF-Encoded data from [13] was only 14%, from [14] was 63%, and from our approach, it was 93.8%. Therefore, there is a $\approx 80\%$ improvement from [13] and $\approx 30\%$ improvement from [14] in the classification accuracy. We experimented with other datasets, and the results followed a similar pattern. Therefore, the dataset encoded with our approach obviously performs well in the DL algorithm compared with the state-of-the-art BF-encoding method. Table. V list the results.

B. The Impact of Different DP-Noise on Data Utility

In order to demonstrate the advantages of using binarized Laplace Noise over the Randomized Response (RR) method, we conducted experiments on the Fashion-MNIST dataset [45]. Initially, we encoded the data in the BF and then applied noise using both methods. The modified encoded datasets were subsequently utilized in classification algorithms. The accuracy of the classification on the original dataset was approximately 91%. However, when we perturbed using the binarized Laplace noise, the accuracy dropped to around 87-88%. Conversely, when RR noise was added, the utility of the BF-encoded data decreased significantly to 75-77%. Our understanding is that in RAPPOR [13], only a few bits (3-4) in the BF are set to 1, while the remaining bits are set to zero. Therefore, when RR is applied to the entire BF, it has minimal impact on the original information. However, in our case, we aimed to enhance the utility of the BF-encoded dataset by having a higher hamming weight [56] (number of bits set to 1 in BF) of around 70-80%. Consequently, when RR noise is introduced, a significant amount of information is modified, thereby reducing the usefulness of the BF-encoded dataset. Taking into account the data utility for downstream tasks, we can conclude that when BF encoding is applied to an image dataset, the Laplacian method outperforms the RR technique. Table. VI list the results.

C. The Impact of Different Hashing Techniques on Attacker Accuracy in MIA

Authors in [14] used the traditional DH technique to map the data in the BF. But, Schnell *et al.* in their research [21] demonstrated that RH can prevent the encoded dataset from data range attacks. Hence, we used the RH method instead

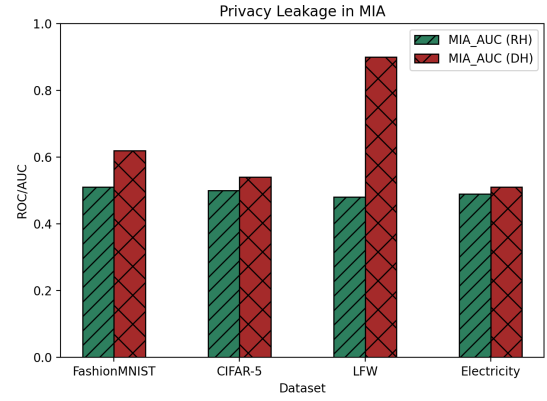


Fig. 7. Privacy Leakage Comparison in Different Hashing Techniques.

of DH to map the data to the Bloom Filter. In the context of our experiment, we assessed the privacy risk of our DL (BloomCNN) model trained using DH-based and RH-based encoded datasets. We observed that the privacy leakage of the DL model increased when trained using the DH-based encoded dataset. Fig. 7 shows the value of Attacker Accuracy (ROC/AUC) generated from the MIA on the DL model trained using the DH and RH encoded dataset. We observed that the information leakage is higher for DH-encoded datasets than compared to RH-encoded datasets. For Fashion-MNIST it increased from 0.51 to 0.62. CIFAR-5 increased from 0.5 to 0.54; for LFW, it increased from 0.48 to 0.9; and for the electricity dataset, it increased from 0.49 to 0.51. Therefore, our proposed differentially private BF-encoding technique ensures more privacy compared to the state-of-the-art techniques.

IX. CONCLUSION

We introduced an improved method of encoding using Bloom Filters, which can be applied to color images, facial images, and time-series datasets. As far as we know, we are the first to demonstrate the usefulness of the Bloom Filter-encoded dataset for DL algorithms. Additionally, we conducted tests to confirm the privacy of the DL model against membership inference attacks. Our results indicated that the model can maintain privacy and can be considered a privacy-preserving DL architecture. In future studies, it would be worth exploring the incorporation of DP-Noise into the class labels, along with the feature encoding, to assess the trade-off between utility and privacy.

APPENDIX A

ATTACKS ON BLOOM FILTER ENCODED DATASET

While the Bloom Filter encoding techniques have shown to be efficient in the privacy-preserving domain, recent research has shown that these techniques can be vulnerable to specific attack methods that can re-identify the sensitive information encoded in the Bloom Filter [36]. Niedermeyer *et al.* [20] proposed a pattern-mining-based cryptanalysis attack on the Bloom Filter by exploiting the weakness of the DH technique. Their attacking approach was manual; Kroll and Steinmetzer [59] extended that as a fully automatic attack. They showed

that the DH represents individual elements of the set into the Bloom Filter by a reasonably small number of possible patterns (in computational and cryptographic terms), which the attacker could identify by initiating a cryptanalysis attack. They [20] discussed that using Random Hashing might be a possible way to prevent their proposed attack. We decided to use this hashing technique in our study.

Vidanage *et al.* in [37] proposed an attack on Bloom Filter that used the measured similarities between encoded records using a graph-based matching technique. In their work, they generated two similarity graphs, one from the encoded dataset and a second one from the plain text data, where the nodes represented the encoded and plain text data, respectively, and edges between the nodes represented the calculated similarity among them. The similarity graphs generated two separate feature matrices for each of the nodes present in those two graphs. Then they applied the locality sensitive hashing (LSH) on the feature matrices to ensure that a group of similar nodes were present in both graphs. They generated a bipartite graph from a similar block of nodes and applied one-to-one matching among them to re-identify the encoded dataset with the plain-text data. The authors showed that if more attribute values of the datasets are encoded, it is not easy to match them with the plain text dataset.

APPENDIX B BINARY NEURAL NETWORK

Binary (or Binarized) Neural Networks (BNNs) [44], [60], [61] are deep neural networks that use binary values for activations and weights. Binary values allow BNNs to execute computations using bitwise operations, reducing execution time. Moreover, the model size is much smaller than its full precision (i.e., 32-bit floating-point values) counterpart [62]. Generally, in BNN, the weights and activations are binarized using a deterministic sign function (Eq. [15]), which constrains the weights and activations between +1 and -1.

$$v_b = \begin{cases} +1, & \text{if } v_f \geq 0 \\ -1, & \text{otherwise} \end{cases} \quad (15)$$

Courbariaux *et al.* [60] mentioned that these two values are very advantageous to speed up computation from a hardware perspective. Moreover, it regularizes the neural network and allows for better performance [62].

These Binary Neural Networks (BNNs) helped us address an open research question related to the usability of BF-Encoded datasets in Deep Learning (DL) algorithms, which consist of multiple layers of neural networks (NNs). Our findings revealed that in BNNs, the input is transformed into +1 and -1 instead of 0 and 1, resulting in enhanced performance of the NN model. Based on this insight, we modified the data representation scheme for the BF, replacing 0 with -1 and 1 with +1 aiming to enhance the effectiveness of the BF-encoded dataset in DL algorithms.

APPENDIX C PRIVACY ANALYSIS

Theorem 1: The addition of Laplace distribution noise thresholded by p -value to a Bloom filter satisfies ϵ -differential privacy where $\epsilon = 2 \cdot k \cdot \ln \left(\frac{1-p}{p} \right)$.

Proof: Let D be the data, BF is the output of Bloom filter from D , and BF' refers to the perturbed BF. Then the probability of observing any given data D assuming that BF' is known, given a raw data element $r \in D$ is:

$$\begin{aligned} P(D = d \mid R = r) &= P(D = d \mid BF, BF', r) \\ &\quad \cdot P(BF' \mid BF, r) \cdot P(BF \mid r) \\ &= P(D = d \mid BF') \cdot P(BF' \mid BF) \cdot P(BF \mid r) \\ &= P(D = d \mid BF') \cdot P(BF' \mid BF). \end{aligned}$$

The first probability in the equation above provides no additional information about BF , as D is conditionally independent of BF , given BF' . However, the second probability ($P(BF' \mid BF)$) has a critical effect on privacy protection. The relevant probabilities can be retrieved as:

$$\begin{aligned} P(bf'_i = 1 \mid bf_i = 1) &= 1 - p, \\ P(bf'_i = 1 \mid bf_i = 0) &= p. \end{aligned}$$

Without loss of generality, let the bits $1, \dots, k$ in the output of the Bloom filter be set to 1, i.e., $bf^* = \{bf_1 = 1, \dots, bf_k = 1, bf_{k+1} = 0, \dots, bf_l = 0\}$. As such,

$$\begin{aligned} P(BF' = bf' \mid BF = bf^*) &= (p)^{bf'_1} (1-p)^{1-bf'_1} \\ &\quad \times \dots \\ &\quad \times (p)^{bf'_k} (1-p)^{1-bf'_k} \times \dots \\ &\quad \times (1-p)^{bf'_{k+1}} (p)^{1-bf'_{k+1}} \times \dots \\ &\quad \times (1-p)^{bf'_l} (p)^{1-bf'_l}. \end{aligned}$$

Let Θ be the ratio of two such conditional probabilities with distinct values of BF , BF_1 and BF_2 ; i.e., $\Theta = \frac{P(BF' \mid BF = BF_1)}{P(BF' \mid BF = BF_2)}$. For the differential privacy condition to hold, Θ needs to be bounded by $\exp(\epsilon)$.

$$\begin{aligned} \Theta &= \frac{P(BF' \mid BF = BF_1)}{P(BF' \mid BF = BF_2)} \\ &= \frac{\sum_{BF'_i \in R} P(BF' = BF'_i \mid BF = BF_1)}{\sum_{BF'_i \in R} P(BF' = BF'_i \mid BF = BF_2)} \\ &\leq \max_{BF'_i \in R} \frac{P(BF' = BF'_i \mid BF = BF_1)}{P(BF' = BF'_i \mid BF = BF_2)} \text{ according to [13]} \\ &= (p)^{2(bf'_1 + bf'_2 + \dots + bf'_k - bf'_{k+1} - bf'_{k+2} - \dots - bf'_{2k})} \\ &\quad \times (1-p)^{2(bf'_{k+1} + bf'_{k+2} + \dots + bf'_{2k} - bf'_1 - bf'_2 - \dots - bf'_k)}. \end{aligned}$$

Sensitivity is maximized when $bf'_{k+1} = bf'_{k+2} = \dots = b'_{2k} = 1$ and $bf'_1 = bf'_2 = \dots = bf'_k = 0$. Then, $\Theta = \left(\frac{1-p}{p} \right)^{2k}$ and $\epsilon = 2 \cdot k \cdot \ln \left(\frac{1-p}{p} \right)$.

TABLE VII
CNN-ARCHITECTURE FOR FASHION-MNIST DATA

Layer	Parameters
Conv-1D	32 Filters of 5x1, ReLU activation
MaxPooling -1D	2x1
Dropout	0.2
Conv-1D	64 Filters of 5x1, ReLU activation
MaxPooling -1D	2x1
Dropout	0.3
Fully Connected	128 Units, ReLU activation
Fully Connected	10 Units, Softmax activation

TABLE VIII
CNN-ARCHITECTURE FOR GENDER-FACE DATA

Layer	Parameters
Conv-1D	32 Filters of 5x1, LeakyReLU activation
BatchNorm	momentum =0.99, epsilon = 0.001
MaxPooling -1D	2x1
Dropout	0.2
Conv-1D	64 Filters of 5x1, LeakyReLU activation
BatchNorm	momentum =0.99, epsilon = 0.001
MaxPooling -1D	2x1
Dropout	0.3
Fully Connected	128 Units, LeakyReLU activation
BatchNorm	momentum =0.99, epsilon = 0.001
Fully Connected	2 Units, Sigmoid activation

APPENDIX D DICE-COEFFICIENT SIMILARITY

The Dice coefficient similarity of two string values x_1 and x_2 using their encoded Bloom filters is calculated as $sim(x_1, x_2) = \frac{2*c}{l_1+l_2}$ where, c is the number of common bit positions that are set to 1 in both BF of x_1 and x_2 , l_1 is the total number of bit positions those are set to 1 in the BF of x_1 and l_2 is the total number of bit positions those are set to 1 in the BF of x_2

APPENDIX E THE CNN-ARCHITECTURE

TABLE IX
CNN-ARCHITECTURE FOR LFW DATA

Layer	Parameters
Conv-1D	32 Filters of 5x1, LeakyReLU activation
MaxPooling -1D	2x1
Dropout	0.2
Conv-1D	64 Filters of 5x1, LeakyReLU activation
MaxPooling -1D	2x1
Dropout	0.3
Fully Connected	128 Units, LeakyReLU activation
Fully Connected	5 Units, Sigmoid activation

TABLE X
CNN-ARCHITECTURE FOR CIFAR-5 DATA

Layer	Parameters
Conv-2D	32 Filters of 3x3, ReLU activation
BatchNorm	momentum =0.99, epsilon = 0.001
Dropout	0.5
Conv-2D	64 Filters of 3x3, ReLU activation
BatchNorm	momentum =0.99, epsilon = 0.001
MaxPooling -2D	2x2
Dropout	0.5
Fully Connected	100 Units, ReLU activation
BatchNorm	momentum =0.99, epsilon = 0.001
Dropout	0.5
Fully Connected	10 Units, Softmax activation

TABLE XI
CNN-ARCHITECTURE FOR FORECASTING

Layer	Parameters
Conv-1D	64 Filters of 3x1, ReLU activation
MaxPooling -1D	2x1
Dropout	0.2
Fully Connected	50 Units, ReLU activation
Fully Connected	1 Units

REFERENCES

- [1] S. Sharma, K. Chen, and A. Sheth, "Toward practical privacy-preserving analytics for iot and cloud-based healthcare systems," *IEEE Internet Computing*, vol. 22, no. 2, pp. 42–51, 2018.
- [2] X. Jiang, X. Zhou, and J. Grossklags, "Privacy-preserving high-dimensional data collection with federated generative autoencoder," *Proc. Priv. Enhancing Technol.*, vol. 2022, no. 1, pp. 481–500, 2022.
- [3] R. Shokri and V. Shmatikov, "Privacy-preserving deep learning," in *Proceedings of the 22nd ACM SIGSAC conference on computer and communications security*, 2015, pp. 1310–1321.
- [4] Q. Song, J. Cao, K. Sun, Q. Li, and K. Xu, "Try before you buy: Privacy-preserving data evaluation on cloud-based machine learning data marketplace," in *Annual Computer Security Applications Conference*, 2021, pp. 260–272.
- [5] D. Zhang, X. Chen, J. Shi, D. Wang, and S. Zeng, "A differential privacy collaborative deep learning algorithm in pervasive edge computing environment," in *2021 IEEE 20th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, IEEE, 2021, pp. 347–354.
- [6] D. Cash, P. Grubbs, J. Perry, and T. Ristenpart, "Leakage-abuse attacks against searchable encryption," in *Proceedings of the 22nd ACM SIGSAC conference on computer and communications security*, 2015, pp. 668–679.
- [7] M. Al-Rubaie and J. M. Chang, "Privacy-preserving machine learning: Threats and solutions," *IEEE Security & Privacy*, vol. 17, no. 2, pp. 49–58, 2019.
- [8] C. Gentry, *A fully homomorphic encryption scheme*. Stanford university, 2009.
- [9] G. Ghinita, P. Karras, P. Kalnis, and N. Mamoulis, "Fast data anonymization with low information loss,"

- in *Proceedings of the 33rd international conference on Very large data bases*, 2007, pp. 758–769.
- [10] C. Dwork, “Differential privacy: A survey of results,” in *International conference on theory and applications of models of computation*, Springer, 2008, pp. 1–19.
 - [11] L. Wang, J. Li, and H. Ahmad, “Challenges of fully homomorphic encryptions for the internet of things,” *IEICE TRANSACTIONS on Information and Systems*, vol. 99, no. 8, pp. 1982–1990, 2016.
 - [12] A. Narayanan and V. Shmatikov, “Robust de-anonymization of large sparse datasets,” in *2008 IEEE Symposium on Security and Privacy (sp 2008)*, IEEE, 2008, pp. 111–125.
 - [13] Ú. Erlingsson, V. Pihur, and A. Korolova, “Rappor: Randomized aggregatable privacy-preserving ordinal response,” in *Proceedings of the 2014 ACM SIGSAC conference on computer and communications security*, 2014, pp. 1054–1067.
 - [14] W. Xue, D. Vatsalan, W. Hu, and A. Seneviratne, “Sequence data matching and beyond: New privacy-preserving primitives based on bloom filters,” *IEEE Transactions on Information Forensics and Security*, vol. 15, pp. 2973–2987, 2020.
 - [15] L. Deng, “The mnist database of handwritten digit images for machine learning research [best of the web],” *IEEE Signal Processing Magazine*, vol. 29, no. 6, pp. 141–142, 2012.
 - [16] D. Anguita, A. Ghio, L. Oneto, X. Parra, J. L. Reyes-Ortiz, *et al.*, “A public domain dataset for human activity recognition using smartphones.” in *Esann*, vol. 3, 2013, p. 3.
 - [17] J. Zhang, B. Wei, W. Hu, and S. S. Kanhere, “Wifi-id: Human identification using wifi signal,” in *2016 International Conference on Distributed Computing in Sensor Systems (DCOSS)*, IEEE, 2016, pp. 75–82.
 - [18] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, “Membership inference attacks against machine learning models,” in *2017 IEEE symposium on security and privacy (SP)*, IEEE, 2017, pp. 3–18.
 - [19] A. Vidanage, T. Ranbaduge, P. Christen, and R. Schnell, “Efficient pattern mining based cryptanalysis for privacy-preserving record linkage,” in *2019 IEEE 35th International Conference on Data Engineering (ICDE)*, IEEE, 2019, pp. 1698–1701.
 - [20] F. Niedermeyer, S. Steinmetzer, M. Kroll, and R. Schnell, “Cryptanalysis of basic bloom filters used for privacy preserving record linkage,” *German Record Linkage Center, Working Paper Series, No. WP-GRLC-2014-04*, 2014.
 - [21] R. Schnell and C. Borgs, “Randomized response and balanced bloom filters for privacy preserving record linkage,” in *2016 IEEE 16th International Conference on Data Mining Workshops (ICDMW)*, IEEE, 2016, pp. 218–224.
 - [22] M. A. *et al.*, *TensorFlow: Large-scale machine learning on heterogeneous systems*, Software available from tensorflow.org, 2015. [Online]. Available: <https://www.tensorflow.org/>.
 - [23] L. Song and P. Mittal, “Systematic evaluation of privacy risks of machine learning models,” in *30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 2615–2632.
 - [24] B. H. Bloom, “Space/time trade-offs in hash coding with allowable errors,” *Communications of the ACM*, vol. 13, no. 7, pp. 422–426, 1970.
 - [25] M. McIlroy, “Development of a spelling list,” *IEEE Transactions on Communications*, vol. 30, no. 1, pp. 91–99, 1982. DOI: 10.1109/TCOM.1982.1095395.
 - [26] A. Z. Broder and M. Mitzenmacher, “Survey: Network applications of bloom filters: A survey,” *Internet Math.*, vol. 1, no. 4, pp. 485–509, 2003.
 - [27] R. Schnell, T. Bachteler, and J. Reiher, “Privacy-preserving record linkage using bloom filters,” *BMC medical informatics and decision making*, vol. 9, no. 1, pp. 1–11, 2009.
 - [28] P. K. Lai, S.-M. Yiu, K. Chow, C. Chong, and L. C. K. Hui, “An efficient bloom filter based solution for multi-party private matching,” in *Security and Management*, Citeseer, 2006, pp. 286–292.
 - [29] D. Vatsalan and P. Christen, “Scalable privacy-preserving record linkage for multiple databases,” in *Proceedings of the 23rd ACM International Conference on Conference on Information and Knowledge Management*, 2014, pp. 1795–1798.
 - [30] D. Vatsalan and P. Christen, “Privacy-preserving matching of similar patients,” *Journal of biomedical informatics*, vol. 59, pp. 285–298, 2016.
 - [31] C. L. Canonne, G. Kamath, and T. Steinke, “The discrete gaussian for differential privacy,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 15 676–15 688, 2020.
 - [32] P. C. Mahawaga Arachchige, D. Liu, S. Camtepe, *et al.*, “Local differential privacy for federated learning,” in *Computer Security—ESORICS 2022: 27th European Symposium on Research in Computer Security, Copenhagen, Denmark, September 26–30, 2022, Proceedings, Part I*, Springer, 2022, pp. 195–216.
 - [33] N. Wang, X. Xiao, Y. Yang, *et al.*, “Collecting and analyzing multidimensional data with local differential privacy,” in *2019 IEEE 35th International Conference on Data Engineering (ICDE)*, IEEE, 2019, pp. 638–649.
 - [34] C. Dwork, F. McSherry, K. Nissim, and A. Smith, “Calibrating noise to sensitivity in private data analysis,” in *Theory of Cryptography: Third Theory of Cryptography Conference, TCC 2006, New York, NY, USA, March 4–7, 2006. Proceedings 3*, Springer, 2006, pp. 265–284.
 - [35] A. Paverd, A. Martin, and I. Brown, “Modelling and automatically analysing privacy properties for honest-but-curious adversaries,” *Tech. Rep.*, 2014.
 - [36] P. Christen, T. Ranbaduge, and R. Schnell, *Linking Sensitive Data*. Springer, 2020.
 - [37] A. Vidanage, P. Christen, T. Ranbaduge, and R. Schnell, “A graph matching attack on privacy-preserving record linkage,” in *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*, 2020, pp. 1485–1494.

- [38] J. Bethencourt, A. Sahai, and B. Waters, "Ciphertext-policy attribute-based encryption," in *2007 IEEE symposium on security and privacy (SP'07)*, IEEE, 2007, pp. 321–334.
- [39] H. Wee, "Optimal broadcast encryption and cp-abe from evasive lattice assumptions," in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Springer, 2022, pp. 217–241.
- [40] B. Ghazi, N. Golowich, R. Kumar, P. Manurangsi, and C. Zhang, "Deep learning with label differential privacy," *Advances in neural information processing systems*, vol. 34, pp. 27 131–27 145, 2021.
- [41] X. Tang, M. Nasr, S. Mahloujifar, *et al.*, "Machine learning with differentially private labels: Mechanisms and frameworks," *Proceedings on Privacy Enhancing Technologies*, vol. 4, pp. 332–350, 2022.
- [42] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [43] N. Papernot, A. Thakurta, S. Song, S. Chien, and Ú. Erlingsson, "Tempered sigmoid activations for deep learning with differential privacy," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, 2021, pp. 9312–9321.
- [44] M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi, "Xnor-net: Imagenet classification using binary convolutional neural networks," in *European conference on computer vision*, Springer, 2016, pp. 525–542.
- [45] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-mnist: A novel image dataset for benchmarking machine learning algorithms," *arXiv preprint arXiv:1708.07747*, 2017.
- [46] A. Krizhevsky, V. Nair, and G. Hinton, "The cifar-10 dataset," *online: <http://www.cs.toronto.edu/kriz/cifar.html>*, vol. 55, no. 5, 2014.
- [47] G. B. Huang, M. Mattar, T. Berg, and E. Learned-Miller, "Labeled faces in the wild: A database for studying face recognition in unconstrained environments," in *Workshop on faces in 'Real-Life' Images: detection, alignment, and recognition*, 2008.
- [48] A. Ponnusamy. "Gender-detection-keras." (2019), [Online]. Available: <https://github.com/arunponnusamy/gender-detection-keras>.
- [49] P. Street, *Dataport pecan street inc*, 2019.
- [50] F. e. a. Pedregosa, "Scikit-learn: Machine learning in python," *the Journal of machine Learning research*, vol. 12, pp. 2825–2830, 2011.
- [51] A. Ponnusamy, *Cvlib -high level computer vision library for python*, 2018. [Online]. Available: <https://pypi.org/project/cvlib/#description>.
- [52] A. Taik and S. Cherkaoui, "Electrical load forecasting using edge computing and federated learning," in *ICC 2020-2020 IEEE International Conference on Communications (ICC)*, IEEE, 2020, pp. 1–6.
- [53] K. Sarker, X. Yang, Y. Li, S. Belkasim, and S. Ji, "A unified plug-and-play framework for effective data denoising and robust abstention," *arXiv preprint arXiv:2009.12027*, 2020.
- [54] L. McInnes, J. Healy, and J. Melville, "Umap: Uniform manifold approximation and projection for dimension reduction," *arXiv preprint arXiv:1802.03426*, 2018.
- [55] A. Kirsch and M. Mitzenmacher, "Less hashing, same performance: Building a better bloom filter," in *European Symposium on Algorithms*, Springer, 2006, pp. 456–467.
- [56] C. Borgs, "Optimal parameter choice for bloom filter-based privacy-preserving record linkage," Ph.D. dissertation, Universität Duisburg-Essen, 2019.
- [57] M. Lin, Q. Chen, and S. Yan, "Network in network," *arXiv preprint arXiv:1312.4400*, 2013.
- [58] X. Zhang, J. Zhao, and Y. LeCun, "Character-level convolutional networks for text classification," *Advances in neural information processing systems*, vol. 28, 2015.
- [59] M. Kroll and S. Steinmetzer, "Automated cryptanalysis of bloom filter encryptions of health records," *arXiv preprint arXiv:1410.6739*, 2014.
- [60] M. Courbariaux, Y. Bengio, and J.-P. David, "Binaryconnect: Training deep neural networks with binary weights during propagations," *Advances in neural information processing systems*, vol. 28, 2015.
- [61] I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, and Y. Bengio, "Binarized neural networks," *Advances in neural information processing systems*, vol. 29, 2016.
- [62] T. Simons and D.-J. Lee, "A review of binarized neural networks," *Electronics*, vol. 8, no. 6, p. 661, 2019.